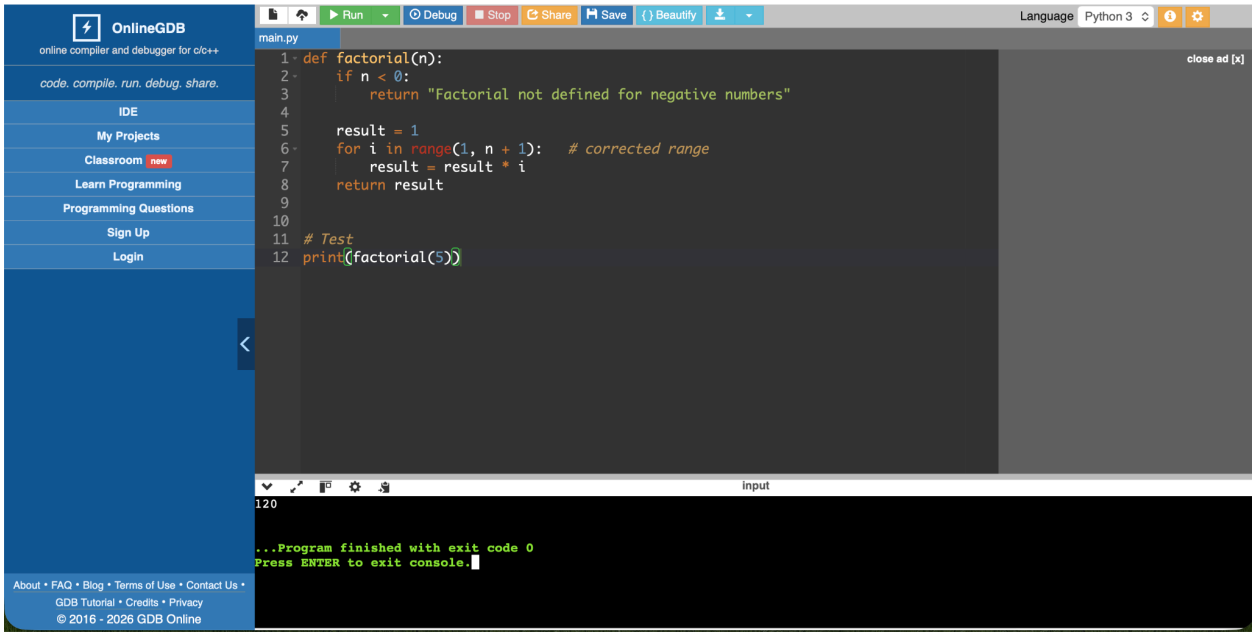
2303A51563
batch=10

Problem Statement 1: AI-Assisted Bug Detection
Scenario: A junior developer wrote the following Python function to calculate factorials:

```
def factorial(n):
result = 1
for i in range(1, n):
result = result * i
return result
```

Instructions:
1. Run the code and test it with factorial(5).
2. Use an AI assistant to:
o Identify the logical bug in the code.
o Explain why the bug occurs (e.g., off-by-one error).
o Provide a corrected version.
3. Compare the AI's corrected code with your own manual fix.
4. Write a brief comparison: Did AI miss any edge cases (e.g., negative numbers, zero)?
Expected Output:
Corrected function should return 120 for factorial(5).
Code

```
def factorial(n):
    if n < 0:
        return "Factorial not defined for negative numbers"

    result = 1
    for i in range(1, n + 1):   # corrected range
        result = result * i
    return result


# Test
print(factorial(5))
```



Problem Statement 2: Task 2 — Improving Readability & Documentation

Scenario:The following code works but is poorly written:

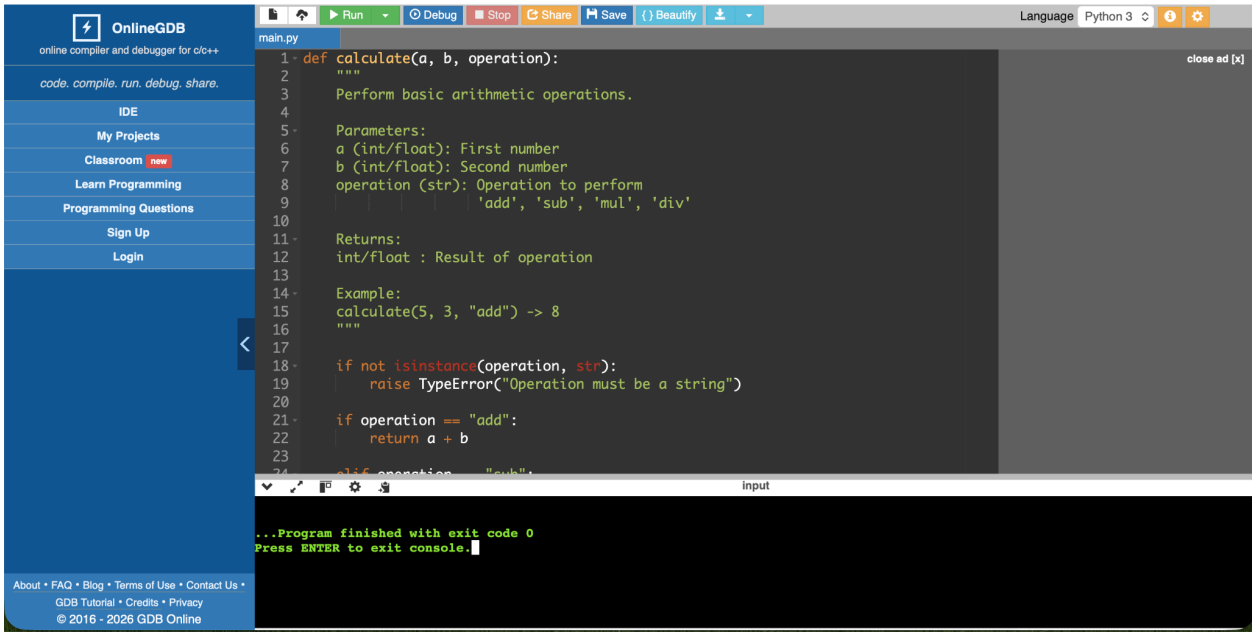
.

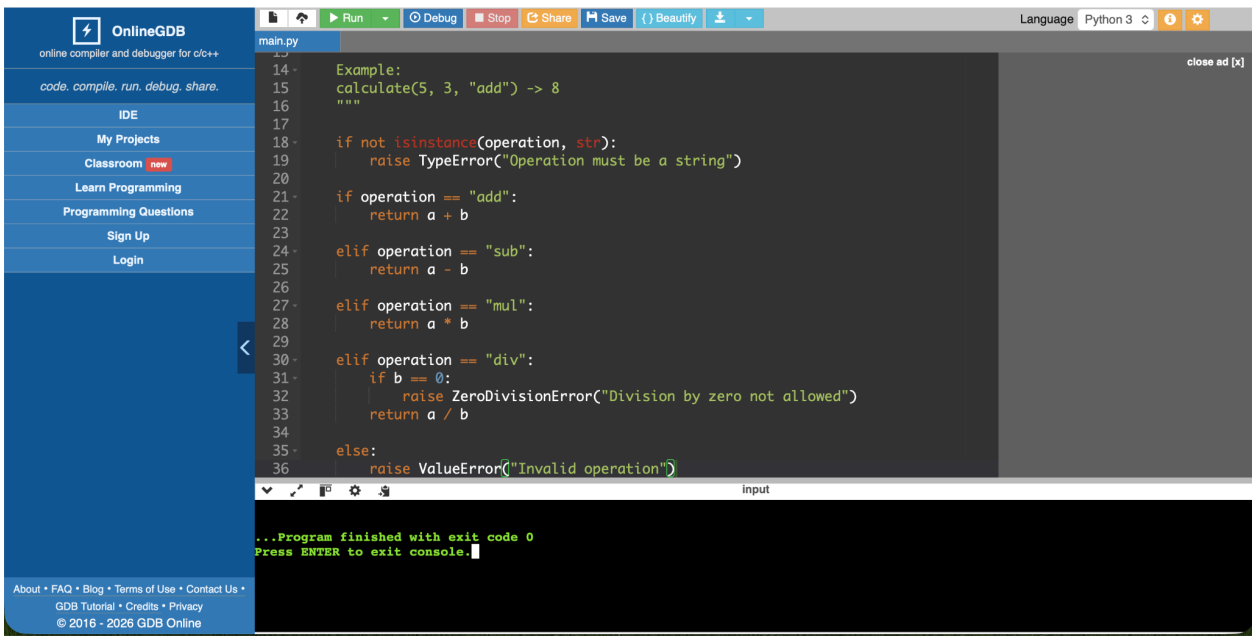
```
def calc(a, b, c):
if c == "add":
return a + b
elif c == "sub":
return a - b
elif c == "mul":
return a * b
elif c == "div":
```

Code

```
def calculate(a, b, operation):
    """
    Perform basic arithmetic operations.

    Parameters:
    a (int/float): First number
    b (int/float): Second number
    operation (str): Operation to perform
                    'add', 'sub', 'mul', 'div'

    Returns:
    int/float : Result of operation

    Example:
    calculate(5, 3, "add") -> 8
```

```
"""

if not isinstance(operation, str):
    raise TypeError("Operation must be a string")

if operation == "add":
    return a + b

elif operation == "sub":
    return a - b

elif operation == "mul":
    return a * b

elif operation == "div":
    if b == 0:
        raise ZeroDivisionError("Division by zero not allowed")
    return a / b

else:
    raise ValueError("Invalid operation")
```





Problem Statement 3: Enforcing Coding Standards

Scenario: A team project requires PEP8 compliance. A developer submits:

```
def Checkprime(n):
for i in range(2, n):
if n % i == 0:
return False
return True
```

Instructions:

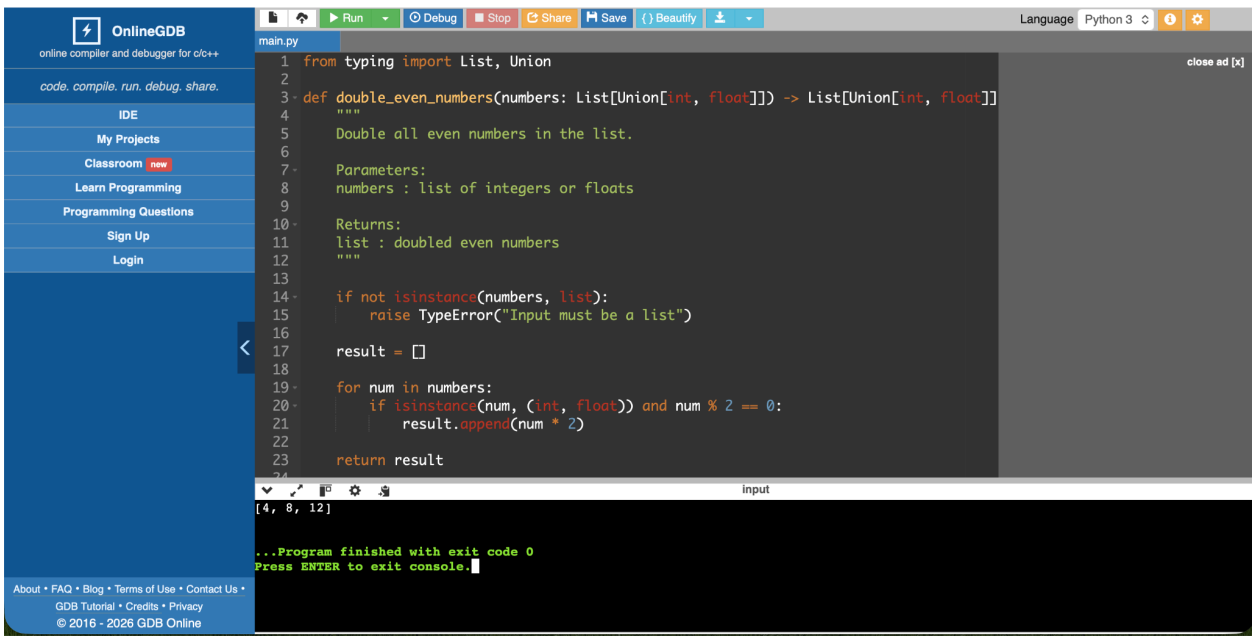
8. Verify the function works correctly for sample inputs.
9. Use an AI tool (e.g., ChatGPT, GitHub Copilot, or a PEP8 linter

with AI explanation) to:
o List all PEP8 violations.
o Refactor the code (function name, spacing, indentation, naming).
10. Apply the AI-suggested changes and verify functionality is preserved.
11. Write a short note on how automated AI reviews could streamline code reviews in large teams.
Expected Output:
A PEP8-compliant version of the function, e.g.:
def check_prime(n):
for i in range(2, n):
if n % i == 0:
return False
return True
Code

```
def check_prime(n):
    """Check whether a number is prime."""

    if n <= 1:
        return False

    for i in range(2, n):
        if n % i == 0:
            return False

    return True
```

# Test
print(check_prime(7))
print(check_prime(10))
Problem Statement 4: AI as a Code Reviewer in Real Projects
Scenario:
In a GitHub project, a teammate submits:
def processData(d):
return [x * 2 for x in d if x % 2 == 0]
Instructions:
1. Manually review the function for:
o Readability and naming.
o Reusability and modularity.
o Edge cases (non-list input, empty list, non-integer
elements).
2. Use AI to generate a code review covering:
a. Better naming and function purpose clarity.
b. Input validation and type hints.
c. Suggestions for generalization (e.g., configurable
multiplier).
3. Refactor the function based on AI feedback.
4. Write a short reflection on whether AI should be a
standalone reviewer or an assistant.
Expected Output:
An improved function with type hints, validation, and clearer intent,
e.g.:
from typing import List, Union
def double_even_numbers(numbers: List[Union[int,
float]]) -> List[Union[int, float]]:
if not isinstance(numbers, list):
raise TypeError("Input must be a list")
return [num * 2 for num in numbers if isinstance(num,
(int, float)) and num % 2 == 0]
Code
```
from typing import List, Union

def double_even_numbers(numbers: List[Union[int, float]]) -> List[Union[int, float]]:
    """
    Double all even numbers in the list.

    Parameters:
    numbers : list of integers or floats

    Returns:
    list : doubled even numbers
```

```
    """

    if not isinstance(numbers, list):
        raise TypeError("Input must be a list")

    result = []

    for num in numbers:
        if isinstance(num, (int, float)) and num % 2 == 0:
            result.append(num * 2)

    return result


# Test
print(double_even_numbers([1,2,3,4,5,6]))
```



Problem Statement 5: — AI-Assisted Performance Optimization

Scenario: You are given a function that processes a list of integers, but it runs slowly on large datasets:

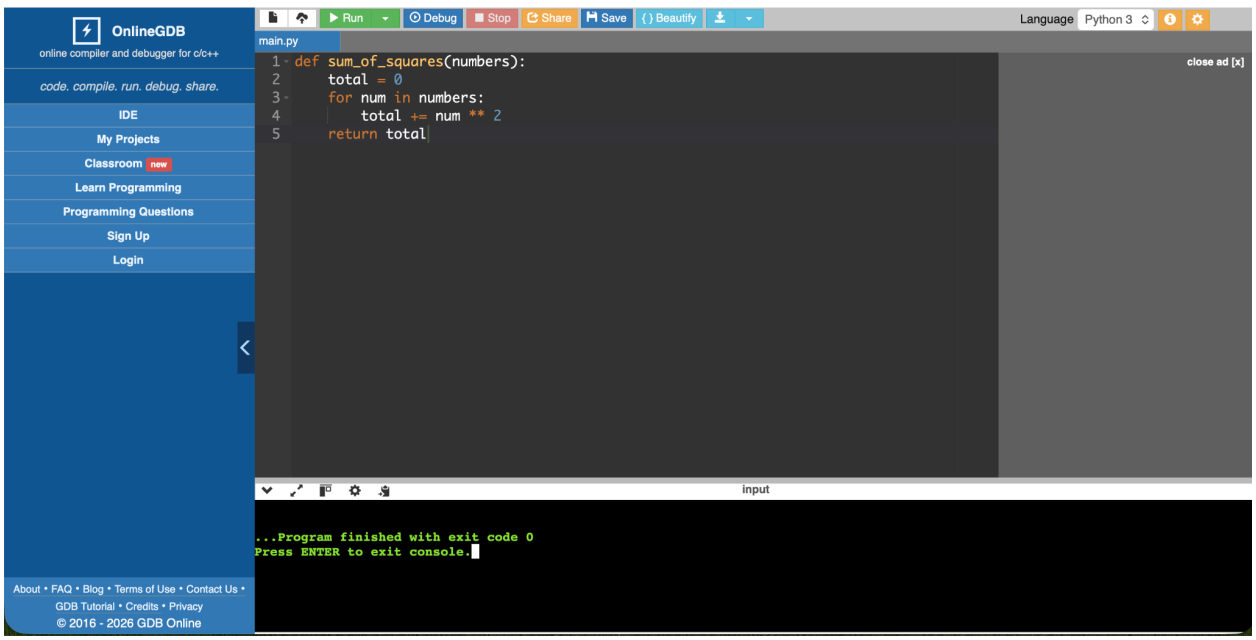
```
def sum_of_squares(numbers):
total = 0
for num in numbers:
total += num ** 2
return total
```

Instructions:

1. Test the function with a large list (e.g., range(1000000)).
2. Use AI to:

o Analyze time complexity.

o Suggest performance improvements (e.g., using built-in functions, vectorization with NumPy if applicable).
o Provide an optimized version.
3. Compare execution time before and after optimization.
4. Discuss trade-offs between readability and performance.
Expected Output:
An optimized function, such as:
def sum_of_squares_optimized(numbers):
return sum(x * x for x in numbers)

Code

```
def sum_of_squares(numbers):
    total = 0
    for num in numbers:
        total += num ** 2
    return total
```